



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών (ΣΗΜΜΥ)

8ο εξάμηνο

Ψηφιακά Συστήματα VLSI

4η εργαστηριακή άσκηση

Δημήτρης Βατάλας

(Α.Μ.: 03121093)

Αριστοτέλης Γεώργιος Δρίτσας

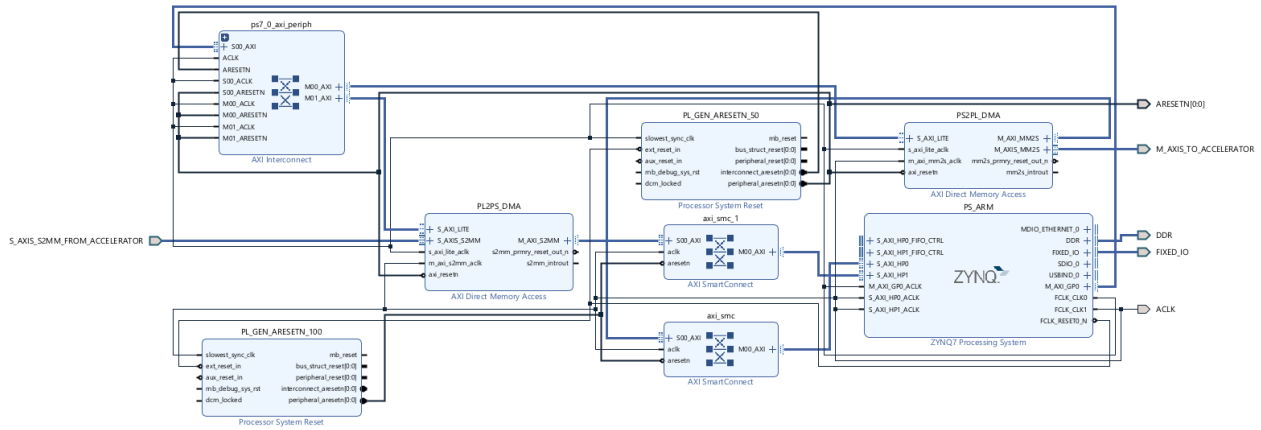
(Α.Μ.: 03122127)

Αθήνα, Απρίλιος 2026

Contents

1	Block Διάγραμμα Συστήματος	2
2	Σύντομη επεξήγηση των βασικών δομικών μονάδων του Κυκλώματος	2
2.1	Calculator	2
2.2	Serial to Parallel	3
2.3	FSM	3
2.4	debayering_filter	3
3	Διάγραμμα fsm	4
4	Ανάλυση χρήσης πόρων για τον FPGA	4
5	Latency	5
6	Throughput	5

1 Block Διάγραμμα Συστήματος



2 Σύντομη επεξήγηση των βασικών δομικών μονάδων του Κυκλώματος

2.1 Calculator

Ο calculator είναι υπεύθυνος για τον υπολογισμό των τιμών R, G και B για κάθε pixel, χρησιμοποιώντας ως είσοδο ένα παράθυρο 3×3 γειτονικών pixels (din00 έως din22), όπως δίνεται ενδεικτικά και στην εκφώνηση.

Ως generic παράμετρος ορίζεται το N , που καθορίζει τη διάσταση της εικόνας προς επεξεργασία. Από την άλλη, η είσοδος του κυκλώματος περιλαμβάνει τα σήματα ελέγχου clk, rst_n και valid_in, dx. Τέλος, η έξοδος αποτελείται από τα υπολογισμένα κανάλια R, G και B, καθώς και το σήμα εγκυρότητας valid_out.

Πρακτικά, για τον υπολογισμό της εξόδου υπάρχει ένας εσωτερικός μηχανισμός που αναγνωρίζει τον τύπο κάθε pixel (GREEN1, GREEN2, BLUE, RED) και ανάλογα με τον τύπο του pixel, γίνεται ο κατάλληλος υπολογισμός της εξόδου σύμφωνα με τη θεωρία του debayering φίλτρου τύπου GBRG.

Τέλος, όσον αφορά τον σωστό χειρισμό του valid_out, όταν το valid_in είναι ίσο με 1 σε έναν κύκλο ρολογιού (και προφανώς δε βρισκόμαστε σε κατάσταση reset, δηλαδή $rst_n = 0$), γνωρίζουμε ότι σε αυτόν τον κύκλο η επεξεργασία είναι έγκυρη και συνεπώς θέτουμε το valid_out επίσης σε 1. Αντίθετα, όταν το valid_in είναι 0, δεν πραγματοποιείται έγκυρος υπολογισμός και το valid_out τίθεται αντίστοιχα σε 0.

2.2 Serial to Parallel

Στη συγκεκριμένη άσκηση το stream των δεδομένων είναι σειριακό. Λόγω όμως της φύσης του debayering, απαιτείται κατάλληλη διαχείριση ώστε να παρέχονται στον calculator τα σωστά pixel τη σωστή χρονική στιγμή. Σε αυτό το σημείο εισάγεται το module serial_to_parallel, το οποίο μετατρέπει το σειριακό stream σε παράλληλη μορφή κατάλληλη για επεξεργασία. Για την υλοποίηση αυτή χρησιμοποιούνται τρεις FIFO ουρές, διατεταγμένες όπως δίνεται ενδεικτικά και στις διαφάνειες, ενώ το 3x3 πλέγμα των δεδομένων που απαιτείται για τον calculator υλοποιείται ουσιαστικά με flip-flops.

Σε κάθε κύκλο ρολογιού, το νέο pixel εισάγεται στην πρώτη FIFO και προωθείται διαδοχικά στις επόμενες, ώστε μέσω των εξόδων των FIFO να σχηματίζεται ένα παράθυρο 3x3. Το παράθυρο αυτό αποτελεί την έγκυρη είσοδο για τον calculator, όπως έχει αναλυθεί προηγουμένως.

Η όλη διαδικασία συγχρονίζεται με έναν μετρητή θέσης pixel, ενώ η έξοδος θεωρείται έγκυρη μόνο όταν έχει σχηματιστεί πλήρως το πρώτο 3x3 παράθυρο επεξεργασίας.

Ως generic παράμετρος ορίζεται το N , που καθορίζει τη διάσταση της εικόνας προς επεξεργασία. Η είσοδος του κυκλώματος περιλαμβάνει τα σήματα `clk`, `rst_n` και `valid_in`, καθώς και το εισερχόμενο pixel stream `din`. Τέλος, η έξοδος αποτελείται από τα στοιχεία της γειτονιάς 3x3 (`dout00`–`dout22`), τα οποία τροφοδοτούν τον calculator, καθώς και το σήμα `valid_out`, το οποίο δηλώνει τότε το παραγόμενο παράθυρο είναι έγκυρο για επεξεργασία.

2.3 FSM

Το συγκεκριμένο module υλοποιεί μια πεπερασμένη μηχανή καταστάσεων (FSM), η οποία διαχειρίζεται τη ροή επεξεργασίας μιας εικόνας από την είσοδο των δεδομένων μέχρι και την ολοκλήρωση της εξόδου. Η λειτουργία της βασίζεται σε έναν εσωτερικό μετρητή και σε διαδοχικές καταστάσεις ελέγχου που καθορίζουν την πορεία του συστήματος.

Ως generic παράμετρος ορίζεται το N , το οποίο καθορίζει τη διάσταση της εικόνας ($N \times N$). Η είσοδος του κυκλώματος περιλαμβάνει τα σήματα `clk`, `rst_n`, `valid_in` και `new_image`, ενώ η έξοδος αποτελείται από το σήμα ενεργοποίησης `en`, τον μετρητή `counter` και το σήμα `image_finished`, το οποίο δηλώνει την ολοκλήρωση της επεξεργασίας.

Πιο συγκεκριμένα, η FSM αποτελείται από πέντε καταστάσεις: `IDLE`, `RECEIVE`, `OUTPUTING`, `SHUTTING_DOWN` και `DONE`. Αρχικά βρίσκεται στην κατάσταση `IDLE` μέχρι να δοθεί νέο image, οπότε και ξεκινά η διαδικασία. Στη συνέχεια, στη `RECEIVE` γίνεται η λήψη των $N \cdot N$ pixel με ταυτόχρονη αύξηση του `counter` όταν το `valid_in` είναι ενεργό. Έπειτα, στην `OUTPUTING` σταματά η λήψη νέων δεδομένων, αλλά συνεχίζονται οι απαραίτητοι υπολογισμοί για τα δεδομένα που βρίσκονται ακόμη μέσα στις FIFO. Στη `SHUTTING_DOWN` υπάρχει για να γίνει ουσιαστικά ένας κύκλος καθυστέρησης (ώστε να βγει το `image_finished` on time) πριν την τελική κατάσταση `DONE`, η οποία σηματοδοτεί την ολοκλήρωση της διαδικασίας.

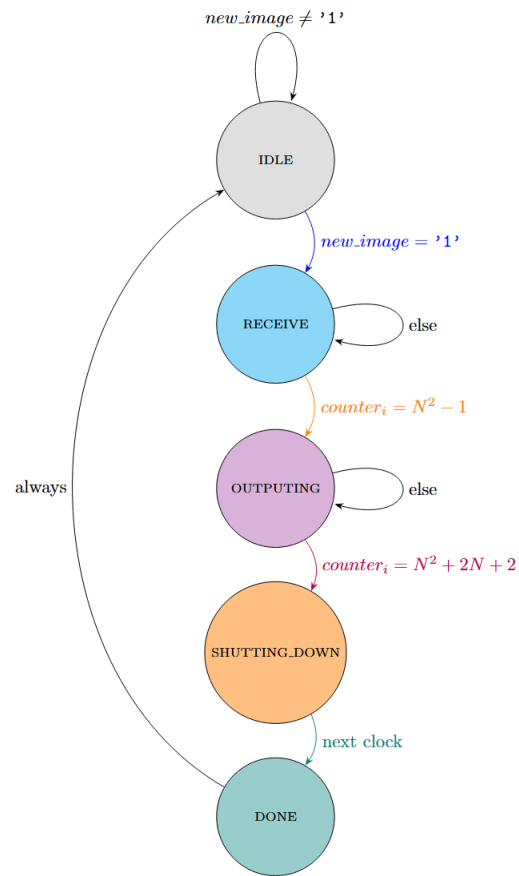
Το σήμα `en` ενεργοποιείται μόνο στις φάσεις `RECEIVE` και `OUTPUTING`, ενώ το `image_finished` τίθεται σε '1' μόνο όταν η FSM ολοκληρώσει τον κύκλο της και βρεθεί στην κατάσταση `DONE`.

2.4 debayering_filter

Το συγκεκριμένο module αποτελεί το top module του συστήματος debayering filter και δεν περιλαμβάνει επιπλέον λογική επεξεργασίας, αλλά λειτουργεί αποκλειστικά ως επίπεδο διασύνδεσης των επιμέρους υπομονάδων που έχουν παρουσιαστεί προηγουμένως.

Ως generic παράμετρος ορίζεται το N , το οποίο καθορίζει τη διάσταση της εικόνας προς επεξεργασία ($N \times N$). Η είσοδος του κυκλώματος περιλαμβάνει τα σήματα `clk`, `rst_n`, `new_image` και `valid_in`, καθώς και το εισερχόμενο pixel stream `pixel`. Αντίστοιχα, η έξοδος αποτελείται από τα κανάλια χρώματος R , G , B , το σήμα `valid_out`, καθώς και το `image_finished` που δηλώνει την ολοκλήρωση της επεξεργασίας.

3 Διάγραμμα fsm



4 Ανάλυση χρήσης πόρων για τον FPGA

- $N = 64$

Resource	Utilization	Available	Utilization %
LUT	3559	17600	20.22
LUTRAM	439	6000	7.32
FF	5196	35200	14.76
BRAM	12.50	60	20.83

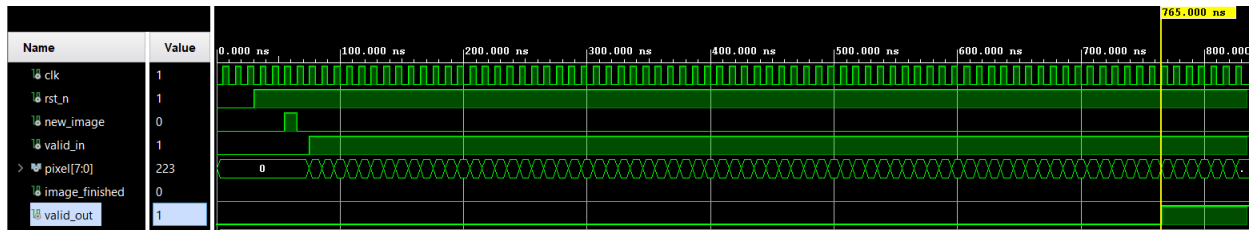
- $N = 128$

Resource	Utilization	Available	Utilization %
LUT	3571	17600	20.29
LUTRAM	438	6000	7.30
FF	5208	35200	14.80
BRAM	12.50	60	20.83

Παρατηρούμε όπως είναι λογικό, ότι για μεγαλύτερο μέγεθος εικόνας έχουμε μια αύξηση στη χρήση πόρων. Το ενδιαφέρον όμως είναι ότι, επειδή υπάρχουν τεχνικές βελτιστοποίησης του design μας κατά το implementation δεν παρατηρούμε τόσο μεγάλη αύξηση όσο θα περιμέναμε διπλασιάζοντας το μέγεθος τριών fifo, αντίθετα βλέπουμε μια σχετικά σταθερή χρήση πόρων, πάρα τον διπλασιασμό του μεγέθους τριών fifo.

5 Latency

Τρέχοντας behavioral simulation ώστε να δούμε την καθυστέρηση από τη στιγμή που έρχεται το πρώτο pixel της εισόδου μέχρι τη στιγμή που θα βγει η πρώτη έγκυρη έξοδος (δηλαδή το latency) έχουμε το παρακάτω:



Παρατηρούμε ότι το πρώτο *valid_out* εμφανίζεται στη χρονική στιγμή 765 ns, ενώ το πρώτο *valid_in* εμφανίστηκε στα 75 ns. Δεδομένου ότι η περίοδος του ρολογιού είναι $T = 10$ ns, προκύπτει καθυστέρηση 68 κύκλων ρολογιού. Το παραπάνω θα μπορούσε να προκύψει και χωρίς να τρέξουμε προσομοίωση δεδομένου ότι έχουμε στον κώδικα μας την παρακάτω γραμμή:

```
valid_out <= '1' when (counter_us > 2*N + 3 and en = '1') else '0';
```

Άρα δεδομένου counter ξεκινά από 0 και για $N = 32$ έχουμε:

$$2N + 3 = 2 \cdot 32 + 3 = 67$$

Άρα το *valid_out* ενεργοποιείται όταν *counter_us* > 67, δηλαδή στον 68ο κύκλο ρολογιού.

6 Throughput

Τέλος, όσον αφορά το throughput παρατηρώ ότι μετά το αρχικό latency, όπως φαίνεται και στο παρακάτω στιγμιότυπο προσομοίωσής, έχουμε ένα σωστό αποτέλεσμα κάθε κύκλο ρολογιού δηλαδή throughput=1

